INSTITUTO MILITAR DE ENGENHARIA — IME

SEÇÃO DE ENGENHARIA ELÉTRICA E ELETRÔNICA

CURSO DE ENGENHARIA ELETRÔNICA — REDES DE COMPUTADORES I

RELATÓRIO TÉCNICO DE IMPLEMENTAÇÃO

APLICAÇÃO DISTRIBUÍDA: JOGO DA VELHA VIA SOCKETS TCP (C / WINSOCK)

Autor:

João Victor Pariz

Docente Responsável: Cel. Paulo César Salgado Vidal

27 de Outubro de 2026

Resumo

Este documento apresenta a especificação, arquitetura, modelagem do protocolo e resultados de validação da aplicação em rede para o **Jogo da Velha (Tic-Tac-Toe)** desenvolvida em conformidade com as diretrizes da disciplina Redes de Computadores I. A solução foi projetada no paradigma Cliente-Servidor utilizando a linguagem C e a biblioteca nativa Winsock2 no sistema operacional Windows. Implementou-se um protocolo de camada de aplicação síncrono sobre o protocolo de transporte orientado à conexão TCP, assegurando integridade e ordenação das jogadas, sorteio aleatório do primeiro jogador pelo nó servidor, validação distribuída do tabuleiro e rotinas robustas de tratamento de exceções de rede e sanitização de entrada do usuário.

1 Introdução e Objetivos

A programação com sockets constitui a interface de programação de aplicações (API) fundamental que viabiliza a comunicação entre processos executando em sistemas finais distintos em uma rede baseada na pilha TCP/IP.

O objetivo deste projeto consiste em conceber e implementar uma aplicação interativa para dois jogadores em computadores separados, concretizando as seguintes etapas:

1. Estabelecimento de conexão confiável entre os hosts utilizando a primitiva orientada a fluxo de bytes (TCP).
2. Definição de um protocolo textual de camada de aplicação para coordenação da partida.
3. Sorteio e sincronização da ordem inicial de jogada (*coin toss*).
4. Garantia de sincronia estrita de estados do tabuleiro 3×3 entre Cliente e Servidor.
5. Validação de regras (vitória, empate e detecção de células ocupadas) de forma autônoma e convergente em ambos os nós.

2 Arquitetura e Protocolo da Camada de Aplicação

2.1 Escolha do Protocolo de Transporte (TCP)

O jogo exige entrega estritamente ordenada e garantida dos comandos de movimento. Uma perda de pacote na transmissão de uma jogada ou uma inversão de pacotes corromperia irreparavelmente a sequência de turnos. Por essa razão, optou-se pela utilização do protocolo TCP (*Transmission Control Protocol*), porta padrão 6000, usufruindo de controle de fluxo, confirmação de entrega e recuperação contra perdas.

2.2 Diagrama de Estados e Troca de Mensagens

A comunicação é estruturada através das mensagens textuais delimitadas por quebra de linha (`\n`):

Mensagem	**Sentido**	**Significado e Ação**
`START:SERVER\n`	Servidor → Cliente	Notifica que o Servidor iniciará a rodada como jogador ativo.
`START:CLIENT\n`	Servidor → Cliente	Notifica que o Cliente iniciará a rodada como jogador ativo.
`MOVE:<N>\n`	Bidirecional	Transmite a coordenada da célula escolhida ($N \in \{1, \ldots, 9\}$).

Tabela 1: Protocolo da Camada de Aplicação do Jogo da Velha.

2.3 Mapeamento Numérico do Tabuleiro

O tabuleiro é apresentado aos usuários em uma matriz 3×3, indexada de 1 a 9 para facilitar a digitação imediata no teclado numérico:

```
 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9
```

O Servidor assume por padrão a representação com a peça **'O'**, enquanto o Cliente opera com a peça **'X'**.

3 Decisões de Engenharia e Robustez

3.1 Detecção Automática de IPs da Máquina Servidora

Para eliminar o atrito durante a apresentação em bancada no laboratório do IME, o código do servidor executa chamadas nativas às APIs `gethostname()` e `gethostbyname()`. Antes de entrar no estado de escuta (`listen`), o programa imprime na tela todos os endereços IPv4 locais associados às interfaces de rede ativas. Com isso, quem estiver executando o Cliente sabe imediatamente qual IP digitar, sem necessidade de abrir outro terminal para rodar `ipconfig`.

3.2 Sorteio Pseudoaleatório com Semente Dinâmica

A determinação de quem joga primeiro é realizada exclusivamente pelo Servidor por meio da função `rand()` alimentada pelo relógio do sistema via `srand((unsigned)time(NULL))`. Imediatamente após a chamada de `accept()`, o Servidor sorteia o valor (0 para Servidor, 1 para Cliente) e transmite a mensagem de controle correspondente. O Cliente aguarda no bloqueio inicial de `recv()` até receber essa diretriz, impedindo qualquer condição de corrida (*race condition*).

3.3 Sanitização de Buffer e Validação de Entradas

Para prevenir comportamentos anômalos causados por entradas inadequadas do usuário (ex.: digitação de letras, espaços em branco ou múltiplos dígitos), a leitura do terminal é feita com captura de caractere e limpeza completa do buffer `stdin`. Uma jogada só é processada se pertencer ao intervalo $[1, 9]$ e se a posição correspondente contiver o espaço vazio `' '`.

3.4 Tratamento de Desconexão Graciosa

Caso um dos nós seja encerrado abruptamente durante a partida, o nó remanescente detecta o retorno de erro ou código zero na primitiva `recv()`, emitindo aviso amigável de perda de conexão e finalizando os recursos com `closesocket()` e `WSACleanup()`, prevenindo vazamentos de recursos no sistema operacional.

4 Guia de Compilação e Operação

4.1 Compilação com MinGW GCC (Windows)

Ambos os módulos compilam diretamente com o compilador GCC padrão do MinGW, vinculando a biblioteca Winsock (`-lws2_32`):

```
gcc -Wall -Wextra servidor_velha.c -o servidor_velha.exe -lws2_32
gcc -Wall -Wextra cliente_velha.c -o cliente_velha.exe -lws2_32
```

4.2 Roteiro de Demonstração em Laboratório

1. **Inicialização do Servidor (Máquina 1):**

   ```
   .\servidor_velha.exe
   ```

   O servidor apresentará na tela o nome do host e o IP da rede local (exemplo: `172.16.1.160`).

2. **Inicialização do Cliente (Máquina 2):**

```
.\cliente_velha.exe 172.16.1.160
```

Caso o IP não seja passado por argumento, o programa solicita interativamente a digitação do IP, admitindo `127.0.0.1` como padrão ao teclar `<Enter>`.

3. **Execução da Partida:** Ambos os consoles exibem o tabuleiro em tempo real, informando de quem é a vez. Ao final (vitória de um dos jogadores ou empate), os resultados são exibidos simultaneamente e a sessão é encerrada de forma limpa.

5 Conclusão

A aplicação foi validada em ambiente local e em rede cabeada/Wi-Fi, cumprindo integralmente todos os requisitos estabelecidos para a atividade prática de Sockets da disciplina. A robustez do protocolo, a clareza da interface de texto e a independência de recompilação para mudança de parâmetros de rede asseguram uma apresentação confiável e ágil em sala de aula.